SQL HACKERRANK QUESTIONS WITH ANSWER

QUESTION 1.

Query all columns for all American cities in the CITY table with populations larger than `100000`. The CountryCode for America is `USA`. The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT * FROM CITY
WHERE POPULATION > 100000 AND CountryCode = 'USA';
```

QUESTION 2.

Query the NAME field for all American cities in the CITY table with populations larger than `120000`. The CountryCode for America is `USA`. The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT NAME FROM CITY
WHERE POPULATION > 120000 AND CountryCode = 'USA';
```

QUESTION 3.

Query all columns (attributes) for every row in the CITY table. The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT * FROM CITY;
```

QUESTION 4.

Query all columns for a city in CITY with the ID `1661`.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT * FROM CITY
WHERE ID = 1661;
```

QUESTION 5.

Query all attributes of every Japanese city in the CITY table. The COUNTRYCODE for Japan is `JPN`. The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT * FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

QUESTION 6.

Query the names of all the Japanese cities in the CITY table. The COUNTRYCODE for Japan is `JPN`. The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT NAME FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

QUESTION 7.

Query a list of CITY and STATE from the STATION table. The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWER:

```
SELECT CITY, STATE FROM STATION;
```

QUESTION 8. (SELECT DISTINCT FUNCTION)

Query a list of CITY names from STATION for cities that have an even ID number. Print the results in any order, but exclude duplicates from the answer. The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

ANSWER:

```
SELECT DISTINCT CITY FROM STATION
WHERE MOD(ID,2)=0 ORDER BY CITY ASC;
```

QUESTION 9. (COUNT & DISTINCT FUNCTION)

Find the difference between the total number of CITY entries in the table and the number of distinct CITY entries in the table. The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude. For example, if there are three records in the table with CITY values 'New York', 'New York', 'Bengalaru', there are 2 different city names: 'New York' and 'Bengalaru'. The query returns , because

Total number of records - number of unique city names = 3 - 2 = 1.

ANSWER:

```
SELECT (COUNT(CITY) - COUNT(DISTINCT CITY)) FROM STATION;
```

—----------

QUESTION 10.

Query the two cities in STATION with the shortest and longest CITY names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.
Sample Input

For example, CITY has four entries: DEF, ABC, PQRS and WXY.
Sample Output

```
ABC 3
```

`PQRS 4`

Explanation

When ordered alphabetically, the **CITY** names are listed as **ABC**, **DEF**, **PQRS**, and **WXY**, with lengths 3,3,4 and 3. The longest name is **PQRS**, but there are options for shortest named city. Choose **ABC**, because it comes first alphabetically.

Note

You can write two separate queries to get the desired output. It need not be a single query.

ANSWER:

```
SELECT CITY C, LENGTH(CITY) L FROM STATION ORDER BY L DESC, C ASC
LIMIT 1;
SELECT CITY C, LENGTH(CITY) L FROM STATION ORDER BY L ASC, C ASC
LIMIT 1;
```

QUESTION 11.

Query the list of CITY names starting with vowels (i.e., `a`, `e`, `i`, `o`, or `u`) from **STATION**. Your result cannot contain duplicates.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Note :

`'a%'` → first letter must be a

`'%a'` → last letter must be a

`'%a%'` → a anywhere

ANSWER:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY LIKE 'A%' OR CITY LIKE
'E%' OR CITY LIKE 'I%' OR CITY LIKE 'O%' OR CITY LIKE 'U%' OR CITY
LIKE 'a%' OR CITY LIKE 'e%' OR CITY LIKE 'i%' OR CITY LIKE 'o%' OR
CITY LIKE 'u%';
```

QUESTION 12.

Query the list of CITY names ending with vowels (i.e., `a`, `e`, `i`, `o`, or `u`) from STATION. Your result cannot contain duplicates.

Input Format: The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Note :

'a%' → first letter must be a

'%a' → last letter must be a

'%a%' → a anywhere

ANSWER:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY LIKE '%A' OR CITY LIKE
'%E' OR CITY LIKE '%I' OR CITY LIKE '%O' OR CITY LIKE '%U' OR CITY
LIKE '%a' OR CITY LIKE '%e' OR CITY LIKE '%i' OR CITY LIKE '%o' OR
CITY LIKE '%u';
```

QUESTION 13.

Query the list of CITY names from STATION which have vowels (i.e., a, e, i, o, and u) as both their first and last characters. Your result cannot contain duplicates.

Input Format : The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT DISTINCT CITY FROM STATION WHERE (CITY LIKE 'A%' OR CITY
LIKE 'E%' OR CITY LIKE 'I%' OR CITY LIKE 'O%' OR CITY LIKE 'U%')
AND (CITY LIKE '%a' OR CITY LIKE '%e' OR CITY LIKE '%i' OR CITY
LIKE '%o' OR CITY LIKE '%u');
```

QUESTION 14.

Query the list of CITY names from STATION that do not start with vowels. Your result cannot contain duplicates.

Input Format: The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT DISTINCT CITY FROM STATION WHERE NOT(CITY LIKE 'A%' OR CITY
LIKE 'E%' OR CITY LIKE 'I%' OR CITY LIKE 'O%' OR CITY LIKE 'U%' OR
CITY LIKE 'a%' OR CITY LIKE 'e%' OR CITY LIKE 'i%' OR CITY LIKE
'o%' OR CITY LIKE 'u%');
```

QUESTION 15.

Query the list of CITY names from STATION that do not end with vowels. Your result cannot contain duplicates.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT DISTINCT CITY FROM STATION WHERE NOT(CITY LIKE '%A' OR CITY LIKE '%E' OR CITY LIKE '%I' OR CITY LIKE '%O' OR CITY LIKE '%U' OR CITY LIKE '%a' OR CITY LIKE '%e' OR CITY LIKE '%i' OR CITY LIKE '%o' OR CITY LIKE '%u');
```

QUESTION 16.

Query the list of CITY names from **STATION** that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT DISTINCT CITY FROM STATION WHERE LOWER(LEFT(CITY,1)) NOT IN ('a','e','i','o','u') OR LOWER(RIGHT(CITY,1)) NOT IN ('a','e','i','o','u');
```

QUESTION 17.

Query the list of CITY names from **STATION** that do not start with vowels and do not end with vowels. Your result cannot contain duplicates.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT DISTINCT CITY FROM STATION WHERE LOWER(LEFT(CITY,1)) NOT IN
('a','e','i','o','u') AND LOWER(RIGHT(CITY,1)) NOT IN
('a','e','i','o','u');
```

QUESTION 18.

Query the Name of any student in **STUDENTS** who scored higher than 75 Marks. Order your output by the last three characters of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending ID.

Input Format: The **STUDENTS** table is described as follows:

Column	*Type*
ID	*Integer*
Name	*String*
Marks	*Integer*

The Name column only contains uppercase (`A`-`Z`) and lowercase (`a`-`z`) letters.

Sample Input:

ID	*Name*	*Marks*
1	*Ashley*	*81*
2	*Samantha*	*75*
4	*Julia*	*76*
3	*Belvet*	*84*

Sample Output

```
Ashley
Julia
Belvet
```

Explanation

Only Ashley, Julia, and Belvet have Marks > 75. If you look at the last three characters of each of their names, there are no duplicates and 'ley' < 'lia' < 'vet'.

ANSWER:

```
SELECT Name FROM STUDENTS WHERE Marks > 75 ORDER BY RIGHT(Name,3),
ID ASC;
```

QUESTION 19.

Write a query that prints a list of employee names (i.e.: the name attribute) from the **Employee** table in alphabetical order.

Input Format: The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where employee_id is an employee's ID number, name is their name, months is the total number of months they've been working for the company, and salary is their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

```
Angela
Bonnie
Frank
Joe
Kimberly
Lisa
Michael
Patrick
Rose
Todd
```

ANSWER:

```
SELECT name FROM Employee ORDER BY name ASC;
```

QUESTION 20.

Write a query that prints a list of employee names (i.e.: the name attribute) for employees in Employee having a salary greater than 2000 per month who have been employees for less than 10 months. Sort your result by ascending employee_id.

Input Format: The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where employee_id is an employee's ID number, name is their name, months is the total number of months they've been working for the company, and salary is the their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

```
Angela
Michael
Todd
Joe
```

Explanation

Angela has been an employee for month and earns per month.

Michael has been an employee for months and earns per month.

Todd has been an employee for months and earns per month.

Joe has been an employee for months and earns per month.

We order our output by ascending employee_id.

ANSWER:

```
SELECT name FROM Employee WHERE (salary > 2000 AND months < 10)
ORDER BY employee_id ASC;
```

QUESTION 21. - Type of Triangle

Write a query identifying the type of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with 3 sides of equal length.
- **Isosceles**: It's a triangle with 2 sides of equal length.

- **Scalene**: It's a triangle with 3 sides of differing lengths.
- **Not A Triangle**: The given values of A, B, and C don't form a triangle.

Input Format

The TRIANGLES table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

Each row in the table denotes the lengths of each of a triangle's three sides.

Sample Input

A	B	C
20	20	23
20	20	20
20	21	22
13	14	30

Sample Output

```
Isosceles
Equilateral
Scalene
Not A Triangle
```

Explanation

Values in the tuple (20, 20, 23) form an Isosceles triangle, because A = B.

Values in the tuple (20, 20, 20) form an Equilateral triangle, because A = B = C.

Values in the tuple (20, 21, 22) form a Scalene triangle, because A ≠ B ≠ C.

Values in the tuple (13, 14, 30) cannot form a triangle because the combined value of sides A and B is not larger than that of side C.

ANSWER:

```
SELECT
CASE
    WHEN A+B<=C OR A+C<=B OR B+C<=A
```

```
        THEN 'Not A Triangle'
    WHEN A=B AND B=C
        THEN 'Equilateral'
    WHEN A=B OR B=C OR C=A
        THEN 'Isosceles'
    ELSE 'Scalene'
END
FROM TRIANGLES;
```

QUESTION 22. - The PADS

Generate the following two result sets:

1. Query an alphabetically ordered list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: `AnActorName(A)`, `ADoctorName(D)`, `AProfessorName(P)`, and `ASingerName(S)`.
2. Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in ascending order, and output them in the following format:

   `There are a total of [occupation_count] [occupation]s.`
   where `[occupation_count]` is the number of occurrences of an occupation in **OCCUPATIONS** and `[occupation]` is the lowercase occupation name. If more than one Occupation has the same `[occupation_count]`, they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.
Input Format: The **OCCUPATIONS** table is described as follows:

Column	*Type*
Name	*String*
Occupation	*String*

Occupation will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.
Sample Input: An **OCCUPATIONS** table that contains the following records:

Name	Occupation
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashely	Professor
Ketty	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

Sample Output

```
Ashely(P)
Christeen(P)
Jane(A)
Jenny(D)
Julia(A)
Ketty(P)
Maria(A)
Meera(S)
Priya(S)
Samantha(D)
There are a total of 2 doctors.
There are a total of 2 singers.
There are a total of 3 actors.
There are a total of 3 professors.
```

Explanation

The results of the first query are formatted to the problem description's specifications. The results of the second query are ascendingly ordered first by number of names corresponding to each profession (2 ≤ 2 ≤ 3 ≤ 3), and then alphabetically by profession (doctor ≤ singer, and actor ≤ professor).

ANSWER:

```
1. SELECT CONCAT(Name,'(',LEFT(Occupation, 1),')') FROM OCCUPATIONS
ORDER BY Name ASC;

2. SELECT CONCAT('There are a total of ',COUNT(Occupation),'
',LOWER(Occupation),'s.') FROM OCCUPATIONS GROUP BY Occupation
ORDER BY COUNT(Occupation), Occupation;
```

QUESTION 23. - Occupations

Pivot the Occupation column in OCCUPATIONS so that each Name is sorted alphabetically and displayed underneath its corresponding Occupation. The output should consist of four columns (Doctor, Professor, Singer, and Actor) in that specific order, with their respective names listed alphabetically under each column.

Note: Print NULL when there are no more names corresponding to an occupation.

Input Format: The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

Occupation will only contain one of the following values: Doctor, Professor, Singer or Actor.

Sample Input

Name	Occupation
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashely	Professor
Ketty	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

Sample Output

Jenny	Ashley	Meera	Jane
Samantha	Christeen	Priya	Julia
NULL	Ketty	NULL	Maria

Explanation

The first column is an alphabetically ordered list of Doctor names.

The second column is an alphabetically ordered list of Professor names.

The third column is an alphabetically ordered list of Singer names.

The fourth column is an alphabetically ordered list of Actor names.

The empty cell data for columns with less than the maximum number of names per occupation (in this case, the Professor and Actor columns) are filled with NULL values.

ANSWER:

```
SELECT
MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
SELECT Name, Occupation, ROW_NUMBER() OVER (PARTITION BY Occupation
ORDER BY Name) AS row_num FROM OCCUPATIONS) AS new_table
GROUP BY row_num;
```

QUESTION 24. - Revising Aggregations - The Count Function

Query a count of the number of cities in CITY having a Population larger than 1,00,00.

Input Format: The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT COUNT(POPULATION) FROM CITY WHERE POPULATION > 100000;
```

QUESTION 25 - Revising Aggregations - The Sum Function

Query the total population of all cities in **CITY** where District is **California**.

Input Format: The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT SUM(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```

QUESTION 26. - Revising Aggregations - Averages

Query the average population of all cities in **CITY** where District is **California**.

Input Format: The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT AVG(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```

QUESTION 27. - Average Population

Query the average population for all cities in **CITY**, rounded down to the nearest integer.

Input Format: The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT FLOOR(AVG(POPULATION)) FROM CITY;
```

QUESTION 28. - Japan Population

Query the sum of the populations for all Japanese cities in CITY. The COUNTRYCODE for Japan is JPN. Input Format: The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT SUM(POPULATION) FROM CITY WHERE COUNTRYCODE = 'JPN';
```

QUESTION 29. - Population Density Difference

Query the difference between the maximum and minimum populations in CITY.

Input Format: The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```

QUESTION 30. - The Blunder

Samantha was tasked with calculating the average monthly salaries for all employees in the EMPLOYEES table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format: The EMPLOYEES table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

ID	Name	Salary
1	Kristeen	1420
2	Ashley	2006
3	Julia	2210
4	Maria	3000

Note: Salary is per month.

Constraints ==> 1000 < Salary < 10^5.

Sample Input (2nd picture)

Sample Output

```
2061
```

Explanation: The table below shows the salaries without zeros as they were entered by Samantha:

ID	Name	Salary
1	Kristeen	142
2	Ashley	26
3	Julia	221
4	Maria	3

Samantha computes an average salary of 98.00. The actual average salary is 2159.00. The resulting error between the two calculations is 2159.00 − 98.00 = 2061.00. Since it is equal to the integer 2061, it does not get rounded up.

ANSWER:

```
SELECT CEIL(AVG(Salary) - AVG(REPLACE(Salary, 0, ''))) FROM
EMPLOYEES;
```

—----------

QUESTION 31. - Top Earners

We define an employee's total earnings to be their monthly salary*monthly worked, and the maximum total earnings to be the maximum total earnings for any employee in the Employee table. Write a query to find the maximum total earnings for all employees as well as the total number of employees who have the maximum total earnings. Then print these values as 2 space-separated integers.

Input Format: The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where employee_id is an employee's ID number, name is their name, months is the total number of months they've been working for the company, and salary is the their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

```
69952 1
```

Explanation

The table and earnings data is depicted in the following diagram:

employee_id	name	months	salary	earnings
12228	Rose	15	1968	29520
33645	Angela	1	3443	3443
45692	Frank	17	1608	27336
56118	Patrick	7	1345	9415
59725	Lisa	11	2330	25630
74197	Kimberly	16	4372	69952
78454	Bonnie	8	1771	14168
83565	Michael	6	2017	12102
98607	Todd	5	3396	16980
99989	Joe	9	3573	32157

The maximum earnings value is 69952. The only employee with earnings = 69952 is Kimberly, so we print the maximum earnings value (69952) and a count of the number of employees who have earned $69952 (which is 1) as two space-separated values.

ANSWER:

NOTE:

`salary * months AS earnings` → **calculates total earnings**

`GROUP BY earnings` → **groups employees who have same earnings**

`COUNT(*)` → **counts employees in each earnings group**

`ORDER BY earnings DESC` → **highest earnings first**

`LIMIT 1` → **takes only the maximum earnings row**

```
SELECT salary*months as earnings, COUNT(*)
FROM Employee
GROUP BY earnings
ORDER BY earnings DESC
LIMIT 1;
```

QUESTION 32. - Weather Observation Station 2

Query the following two values from the STATION table:

1. The sum of all values in LAT_N rounded to a scale of 2 decimal places.
2. The sum of all values in LONG_W rounded to a scale of 2 decimal places.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Output Format

Your results must be in the form:

```
lat lon
```

Where lat is the sum of all values in LAT_N and lon is the sum of all values in LONG_W. Both results must be rounded to a scale of 2 decimal places.

ANSWER:

```
SELECT ROUND(SUM(LAT_N), 2), ROUND(SUM(LONG_W), 2) FROM STATION;
```

QUESTION 33. - Weather Observation Station 13

Query the sum of Northern Latitudes (LAT_N) from **STATION** having values greater than `38.7880` and less than `137.2345`. Truncate your answer to 4 decimal places.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT TRUNCATE(SUM(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7880
```

```
AND LAT_N < 137.2345;
```

QUESTION 34. - Weather Observation Station 14

Query the greatest value of the Northern Latitudes (LAT_N) from **STATION** that is less than `137.2345`. Truncate your answer to 4 decimal places.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT TRUNCATE(MAX(LAT_N), 4) FROM STATION WHERE LAT_N < 137.2345;
```

QUESTION 35. - Weather Observation Station 15

Query the Western Longitude (LONG_W) for the largest Northern Latitude (LAT_N) in **STATION** that is less than `137.2345`. Round your answer to 4 decimal places.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER: `SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N < 137.2345 ORDER BY LAT_N DESC LIMIT 1;`

QUESTION 36. - Weather Observation Station 16

Query the smallest Northern Latitude (LAT_N) from **STATION** that is greater than `38.7880`.

Round your answer to 4 decimal places.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT ROUND(MIN(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7780;
```

QUESTION 37. - Weather Observation Station 17

Query the Western Longitude (LONG_W)where the smallest Northern Latitude (LAT_N) in **STATION** is greater than `38.7880`. Round your answer to 4 decimal places.

Input Format: The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N > 38.7780 ORDER BY
LAT_N ASC LIMIT 1;
```

QUESTION 38. - Population Census

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the

CONTINENT is 'Asia'.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format: The CITY and COUNTRY tables are described as follows:

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT SUM(CITY.POPULATION) FROM CITY INNER JOIN COUNTRY ON
CITY.COUNTRYCODE = COUNTRY.CODE WHERE COUNTRY.CONTINENT = 'Asia';
```

QUESTION 39. - African Cities

Given the CITY and COUNTRY tables, query the names of all cities where the CONTINENT is 'Africa'.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format: The CITY and COUNTRY tables are described as follows:

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT CITY.NAME FROM CITY INNER JOIN COUNTRY ON CITY.COUNTRYCODE =
COUNTRY.CODE WHERE COUNTRY.CONTINENT = 'Africa';
```

QUESTION 40. - Average Population of Each Continent

Given the CITY and COUNTRY tables, query the names of all the continents (COUNTRY.Continent) and their respective average city populations (CITY.Population) rounded down to the nearest integer.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format: The CITY and COUNTRY tables are described as follows:

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

ANSWER:

```
SELECT co.CONTINENT, FLOOR(AVG(c.POPULATION))FROM CITY AS c INNER
JOIN COUNTRY AS co ON c.COUNTRYCODE = co.CODE GROUP BY
co.CONTINENT;
```

QUESTION 41. - Draw The Triangle 1

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern P(20).

ANSWER:

```
-- METHOD 1
WITH RECURSIVE numbers AS (
      SELECT 20 AS n
      UNION ALL
      SELECT n - 1
      FROM numbers
      WHERE n > 1
)
SELECT REPEAT('* ', n) FROM numbers;
-- METHOD 2 WHILE LOOP
DELIMITER $$
CREATE PROCEDURE while_lp()
BEGIN
      DECLARE i INT DEFAULT 20;
      WHILE i > 0 DO
      SELECT REPEAT('* ', i);
      SET i = i - 1;
      END WHILE;
END $$
```

```
DELIMITER ;
CALL while_lp();
```

QUESTION 42. - Draw The Triangle 2

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern P(20).

ANSWER:

```
DELIMITER $$
CREATE PROCEDURE while_lp()
BEGIN
     DECLARE j INT DEFAULT 1;
     WHILE j < 21 DO
     SELECT REPEAT('* ', j);
     SET j = j + 1;
     END WHILE;
END $$
DELIMITER ;
CALL while_lp();
```

QUESTION 43 - Weather Observation Station 18

Consider P1(a,b) and P2(c,d) to be two points on a 2D plane.

- **a happens to equal the minimum value in Northern Latitude (LAT_N in STATION).**
- **b happens to equal the minimum value in Western Longitude (LONG_W in STATION).**
- **c happens to equal the maximum value in Northern Latitude (LAT_N in STATION).**
- **d happens to equal the maximum value in Western Longitude (LONG_W in STATION).**

Query the Manhattan Distance between points P1 and P2 and round it to a scale of 4 decimal places.

Input Format: The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

Note: ABS(number) = Always Positive Value

The question says: Find the Manhattan Distance

Manhattan Distance Formula

For two points:

(x1, y1) and (x2, y2)

Formula:

|x1 - x2| + |y1 - y2|

See those vertical lines | | ?

They mean:

Absolute value

So whenever you see:

| something |

👉 That means use ABS( ) in SQL.

```
SELECT ROUND( ABS(MIN(LAT_N)-MAX(LAT_N)) +
ABS(MIN(LONG_W)-MAX(LONG_W)), 4) FROM STATION;
```

—----------

QUESTION 44. - Binary Tree Nodes

You are given a table, BST, containing two columns: N and P, where N represents the value of a node in Binary Tree, and P is the parent of N.

Column	*Type*
N	*Integer*
P	*Integer*

Write a query to find the node type of Binary Tree ordered by the value of the node. Output one of the following for each node:

- **Root: If node is root node.**
- **Leaf: If node is leaf node.**
- **Inner: If node is neither root nor leaf node.**

Sample Input

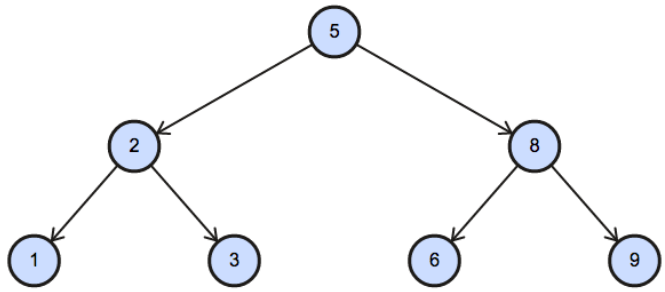
N	P
1	2
3	2
6	8
9	8
2	5
8	5
5	null

Sample Output

```
1 Leaf
2 Inner
3 Leaf
5 Root
6 Leaf
8 Inner
9 Leaf
```

Explanation

The Binary Tree below illustrates the sample:



ANSWER:

```
SELECT N,
    CASE
     WHEN P IS NULL THEN 'Root'
     WHEN N NOT IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN
'Leaf'
     ELSE 'Inner'
    END
FROM BST
ORDER BY N;
```

QUESTION 45. - New Companies

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:

Founder
⇓
Lead Manager
⇓
Senior Manager
⇓
Manager
⇓
Employee

Given the table schemas below, write a query to print the company_code, founder name, total number of lead managers, total number of senior managers, total number of managers, and total number of employees. Order your output by ascending company_code.

Note:

- **The tables may contain duplicate records.**
- **The company_code is string, so the sorting should not be numeric. For example, if the company_codes are C_1, C_2, and C_10, then the ascending company_codes will be C_1, C_10, and C_2.**

Input Format: The following tables contain company data:

- **Company: The company_code is the code of the company and founder is the founder of the company.**

Column	Type
company_code	String
founder	String

- **Lead_Manager: The lead_manager_code is the code of the lead manager, and the company_code is the code of the working company.**

Column	Type
lead_manager_code	String
company_code	String

- **Senior_Manager: The senior_manager_code is the code of the senior manager, the lead_manager_code is the code of its lead manager, and the company_code is the code of the working company.**

Column	Type
senior_manager_code	String
lead_manager_code	String
company_code	String

- **Manager: The manager_code is the code of the manager, the senior_manager_code is the code of its senior manager, the lead_manager_code is the code of its lead manager, and the company_code is the code of the working company.**

Column	Type
manager_code	String
senior_manager_code	String
lead_manager_code	String
company_code	String

- **Employee: The employee_code is the code of the employee, the manager_code is the code of its manager, the senior_manager_code is the code of its senior manager, the lead_manager_code is the code of its lead manager, and the**

company_code is the code of the working company.

Column	Type
employee_code	String
manager_code	String
senior_manager_code	String
lead_manager_code	String
company_code	String

Sample Input:

Company Table:

company_code	founder
C1	Monika
C2	Samantha

Lead_Manager Table:

lead_manager_code	company_code
LM1	C1
LM2	C2

Senior_Manager Table:

senior_manager_code	lead_manager_code	company_code
SM1	LM1	C1
SM2	LM1	C1
SM3	LM2	C2

Manager Table:

manager_code	senior_manager_code	lead_manager_code	company_code
M1	SM1	LM1	C1
M2	SM3	LM2	C2
M3	SM3	LM2	C2

Employee Table:

employee_code	manager_code	senior_manager_code	lead_manager_code	company_code
E1	M1	SM1	LM1	C1
E2	M1	SM1	LM1	C1
E3	M2	SM3	LM2	C2
E4	M3	SM3	LM2	C2

Sample Output

```
C1 Monika 1 2 1 2
C2 Samantha 1 1 2 2
```

Explanation

In company C1, the only lead manager is LM1. There are two senior managers, SM1 and SM2, under LM1. There is one manager, M1, under senior manager SM1. There are two employees, E1 and E2, under manager M1.In company C2, the only lead manager is LM2. There is one senior manager, SM3, under LM2. There are two managers, M2 and M3, under senior manager SM3. There is one employee, E3, under manager M2, and another employee, E4, under manager, M3.

ANSWER:

```sql
-- METHOD 1
SELECT c.company_code, c.founder,
COUNT(DISTINCT e.lead_manager_code),
COUNT(DISTINCT e.senior_manager_code),
COUNT(DISTINCT e.manager_code),
COUNT(DISTINCT e.employee_code)
FROM Employee e
INNER JOIN Company c
ON e.company_code = c.company_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code ASC;

-- METHOD 2
SELECT
    C.company_code,
    C.founder,
    COUNT(DISTINCT lm.lead_manager_code),
    COUNT(DISTINCT sm.senior_manager_code),
    COUNT(DISTINCT m.manager_code),
    COUNT(DISTINCT e.employee_code)
FROM Company c
LEFT JOIN Lead_Manager lm
    ON c.company_code = lm.company_code
LEFT JOIN Senior_Manager sm
    ON c.company_code = sm.company_code
```

```sql
LEFT JOIN Manager m
    ON c.company_code = m.company_code
LEFT JOIN Employee e
    ON c.company_code = e.company_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code;
```

BIG DIFFERENCE

Feature	Method 1	Method 2
Starts from	Employee	Company
Uses	INNER JOIN	LEFT JOIN
Shows company with 0 employees	No	Yes
Interview safe	Risky	Yes
Logically complete	Not fully	Fully

—----------

QUESTION 46. - Weather Observation Station 19

Consider P1(a,c) and P2(b,d) to be two points on a 2D plane where (a,b) are the respective minimum and maximum values of Northern Latitude (LAT_N) and (c,d) are the respective minimum and maximum values of Western Longitude (LONG_W) in STATION.

Query the Euclidean Distance between points P1 and P2 and format your answer to display 4 decimal digits.

Input Format: The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

√((x2 - x1)^2 + (y2 - y1)^2) - Euclidean Distance formula

x1 = MIN(LAT_N), y1 = MIN(LONG_W), x2 = MAX(LAT_N), y2 = MAX(LONG_W)

```
SELECT
    ROUND(
        SQRT(
            POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
            POWER(MAX(LONG_W) - MIN(LONG_W), 2)
        )
    , 4)
FROM STATION;
```

—----------

QUESTION 47. - Weather Observation Station 20

A median is defined as a number separating the higher half of a data set from the lower half. Query the median of the Northern Latitudes (LAT_N) from STATION and round your answer to 4 decimal places.

Input Format: The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

ANSWER:

```
SELECT ROUND(AVG(LAT_N), 4)
FROM (
    SELECT LAT_N,
        ROw_NUMBER() OVER(ORDER BY LAT_N) AS row_num,
        COUNT(*) OVER() AS total_count
    FROM STATION
) AS sub
WHERE row_num IN (
    FLOOR((total_count + 1) / 2),
    FLOOR((total_count + 2) / 2)
);
```

##

Step-by-Step Explanation

Step 1: Inner Query

SELECT LAT_N, ROW_NUMBER() OVER (ORDER BY LAT_N) AS rn, COUNT(*) OVER () AS total_count FROM STATION

What happens here?

`ROW_NUMBER() OVER (ORDER BY LAT_N)`

- Sorts `LAT_N` in ascending order.
- Assigns row numbers starting from 1.

Example (after sorting):

LAT_N	rn
10.1	1
15.3	2
20.5	3
25.7	4
30.9	5

```
COUNT(*) OVER () AS total_count
```

- Counts total rows.
- Same value is shown in every row.

If total rows = 5:

LAT_N	rn	total_count
10.1	1	5
15.3	2	5
20.5	3	5
25.7	4	5
30.9	5	5

This full result is stored as temp table.

Step 2: Finding Middle Position

```
WHERE rn IN (
  FLOOR((total_count + 1) / 2),
  FLOOR((total_count + 2) / 2)
)
```

This is the most important part.

🔹 Case 1: Odd number of rows

If total_count = 5

(total_count + 1) / 2 = (5 + 1) / 2 = 3

(total_count + 2) / 2 = (5 + 2) / 2 = 3.5 → FLOOR = 3

So:

rn IN (3, 3)

→ Only row number 3 is selected

→ That is the median.

🔹 Case 2: Even number of rows

If total_count = 6

(total_count + 1) / 2 = (6 + 1) / 2 = 3.5 → FLOOR = 3

(total_count + 2) / 2 = (6 + 2) / 2 = 4

So:

rn IN (3, 4)

→ Rows 3 and 4 are selected

→ These are the two middle values.

Step 3: Outer Query

SELECT ROUND(AVG(LAT_N), 4)

Why `AVG()`?

- If odd → only one value selected → AVG returns same value.
- If even → two middle values selected → AVG calculates their average.

Finally:

- `ROUND(..., 4)` → rounds result to 4 decimal places.

##

QUESTION 48. - The Report

You are given two tables: Students and Grades. Students contains three columns ID, Name and Marks.

Column	Type
ID	Integer
Name	String
Marks	Integer

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives Eve a task to generate a report containing three columns: Name, Grade and Mark. Ketty doesn't want the NAMES of those students who received a grade lower than 8. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order. Write a query to help Eve.

Sample Input

ID	Name	Marks
1	Julia	88
2	Samantha	68
3	Maria	99
4	Scarlet	78
5	Ashley	63
6	Jane	81

Sample Output

```
Maria 10 99
Jane 9 81
```

`Julia 9 88`
`Scarlet 8 78`
`NULL 7 63`
`NULL 7 68`

Note: Print "NULL" as the name if the grade is less than 8.

Explanation: Consider the following table with the grades assigned to the students:

ID	Name	Marks	Grade
1	Julia	88	9
2	Samantha	68	7
3	Maria	99	10
4	Scarlet	78	8
5	Ashley	63	7
6	Jane	81	9

So, the following students got 8, 9 or 10 grades:

- Maria (grade 10)
- Jane (grade 9)
- Julia (grade 9)
- Scarlet (grade 8)

ANSWER:

```sql
SELECT
    CASE
        WHEN g.Grade < 8 THEN NULL
        ELSE s.Name
    END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
    ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
    CASE WHEN g.Grade >= 8 THEN s.Name END ASC,
    CASE WHEN g.Grade < 8 THEN s.Marks END ASC;
```

—----------

QUESTION 49. - Top Competitors

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective hacker_id and name of hackers who achieved full scores for more than one challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending hacker_id.

Input Format : **The following tables contain contest data:**

- **Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.**

Column	Type
hacker_id	Integer
name	String

- **Difficulty: The difficult_level is the level of difficulty of the challenge, and score is the maximum score that can be achieved for a challenge at that difficulty level.**

Column	Type
difficulty_level	Integer
score	Integer

- **Challenges: The challenge_id is the id of the challenge, the hacker_id is the id of the hacker who created the challenge, and difficulty_level is the level of difficulty of the challenge.**

Column	Type
challenge_id	Integer
hacker_id	Integer
difficulty_level	Integer

- **Submissions: The submission_id is the id of the submission, hacker_id is the id of the hacker who made the submission, challenge_id is the id of the challenge that**

the submission belongs to, and score is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Sample Input

Hackers Table:

hacker_id	name
5580	Rose
8439	Angela
27205	Frank
52243	Patrick
52348	Lisa
57645	Kimberly
77726	Bonnie
83082	Michael
86870	Todd
90411	Joe

Difficulty Table:

difficulty_level	score
1	20
2	30
3	40
4	60
5	80
6	100
7	120

Challenges Table:

challenge_id	hacker_id	difficulty_level
4810	77726	4
21089	27205	1
36566	5580	7
66730	52243	6
71055	52243	2

Submissions Table:

submission_id	hacker_id	challenge_id	score
68628	77726	36566	30
65300	77726	21089	10
40326	52243	36566	77
8941	27205	4810	4
83554	77726	66730	30
43353	52243	66730	0
55385	52348	71055	20
39784	27205	71055	23
94613	86870	71055	30
45788	52348	36566	0
93058	86870	36566	30
7344	8439	66730	92
2721	8439	4810	36
523	5580	71055	4
49105	52348	66730	0
55877	57645	66730	80
38355	27205	66730	35
3924	8439	36566	80
97397	90411	66730	100
84162	83082	4810	40
97431	90411	71055	30

Sample Output

```
90411 Joe
```

Explanation

Hacker 86870 got a score of 30 for challenge 71055 with a difficulty level of 2, so 86870 earned a full score for this challenge.

Hacker 90411 got a score of 30 for challenge 71055 with a difficulty level of 2, so 90411 earned a full score for this challenge.

Hacker 90411 got a score of 100 for challenge 66730 with a difficulty level of 6, so 90411 earned a full score for this challenge.

Only hacker 90411 managed to earn a full score for more than one challenge, so we print the their hacker_id and name as 2 space-separated values.

ANSWER:

```
SELECT h.hacker_id, h.name
    FROM Hackers h
        JOIN Submissions s
    ON h.hacker_id = s.hacker_id
        JOIN Challenges c
    ON s.challenge_id = c.challenge_id
        JOIN Difficulty d
    ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```

Step-by-step explanation

Step 1: Tables Used in the Problem

1. Hackers ->Stores hacker details.
2. Challenges -> Stores who created the challenge.
3. Difficulty -> Stores the full score for each difficulty level.
4. Submissions -> Stores scores hackers got for challenges.

Step 2: What the Question Wants

We must print:

`Hacker_id, name` ONLY if:

✔ The hacker **got full score** ✔ In **more than 1 challenge**

Step:3 What is "Full Score"?

Full score means:

submission.score = difficulty.score

Step 4 : Join Tables

We must connect the tables.

Relations:

Hackers.hacker_id = Submissions.hacker_id

Submissions.challenge_id = Challenges.challenge_id

Challenges.difficulty_level = Difficulty.difficulty_level

Step 5: Filter Only Full Scores

We only want rows where hacker got full score.

HERE s.score = d.score

Step 6: Count Full Score Challenges

We count how many full scores each hacker got.

COUNT(s.challenge_id)

But counting must be done per hacker, so we use:

GROUP BY h.hacker_id, h.name

Step 7: Only Hackers With More Than 1 Full Score

We filter using HAVING.

HAVING COUNT(s.challenge_id) > 1

Step 8: Sorting

The question says:

1. Descending by number of full scores

    ORDER BY COUNT(s.challenge_id) DESC

2. If tie → ascending hacker_id

    , h.hacker_id ASC

QUESTION 50. - Ollivander's Inventory

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each non-evil wand of high power and age. Write a query to print the id, age, coins_needed, and power of the wands that Ron's interested in, sorted in order of descending power. If more than one wand has same power, sort the result in order of descending age.

Input Format: The following tables contain data on the wands in Ollivander's inventory:

- **Wands: The id is the id of the wand, code is the code of the wand, coins_needed is the total number of gold galleons needed to buy the wand, and power denotes the**

quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

- **Wands_Property: The code is the code of the wand, age is the age of the wand, and is_evil denotes whether the wand is good for the dark arts. If the value of is_evil is 0, it means that the wand is not evil. The mapping between code and age is one-one, meaning that if there are two pairs, (code1, age1) and (code2, age2), then code1 ≠ code2 and age1 ≠ age2.**

Column	Type
code	Integer
age	Integer
is_evil	Integer

Sample Input **Wands Table and Wands_Property Table:**

id	code	coins_needed	power
1	4	3688	8
2	3	9365	3
3	3	7187	10
4	3	734	8
5	1	6020	2
6	2	6773	7
7	3	9873	9
8	3	7721	7
9	1	1647	10
10	4	504	5
11	2	7587	5
12	5	9897	10
13	3	4651	8
14	2	5408	1
15	2	6018	7
16	4	7710	5
17	2	8798	7
18	2	3312	3
19	4	7651	6
20	5	5689	3

code	age	is_evil
1	45	0
2	40	0
3	4	1
4	20	0
5	17	0

Sample Output

```
9 45 1647 10
12 17 9897 10
1 20 3688 8
15 40 6018 7
19 20 7651 6
11 40 7587 5
10 20 504 5
18 40 3312 3
20 17 5689 3
5 45 6020 2
14 40 5408 1
```

Explanation:

The data for wands of age 45 (code 1):

id	age	coins_needed	power
5	45	6020	2
9	45	1647	10

• The minimum number of galleons needed for wand(age = 45, power = 2) = 6020
• The minimum number of galleons needed for wand(age = 45, power = 10) = 1647

The data for wands of age 40 (code 2):

id	age	coins_needed	power
14	40	5408	1
18	40	3312	3
11	40	7587	5
15	40	6018	7
17	40	8798	7
6	40	6773	7

• The minimum number of galleons needed for wand(age = 40, power = 1) = 5408
• The minimum number of galleons needed for wand(age = 40, power = 3) = 3312
• The minimum number of galleons needed for wand(age = 40, power = 5) = 7587
• The minimum number of galleons needed for wand(age = 40, power = 7) = 6018

The data for wands of age 20 (code 4):

id	age	coins_needed	power
10	20	504	5
16	20	7710	5
19	20	7651	6
1	20	3688	8

• The minimum number of galleons needed for wand(age = 20, power = 5) = 504
• The minimum number of galleons needed for wand(age = 20, power = 6) = 7651
• The minimum number of galleons needed for wand(age = 20, power = 8) = 3688

The data for wands of age 17 (code 5):

id	age	coins_needed	power
20	17	5689	3
12	17	9897	10

• The minimum number of galleons needed for wand(age = 17, power = 3) = 5689
• The minimum number of galleons needed for wand(age = 17, power = 10) = 9897

ANSWER:

```
SELECT
    w.id,
    Wp.age,
    W.coins_needed,
    W.power
FROM Wands AS w
JOIN Wands_Property AS wp
ON w.code = wp.code
WHERE wp.is_evil = 0
    AND w.coins_needed = (
        SELECT MIN(w2.coins_needed)
        FROM Wands AS w2
            JOIN Wands_Property AS wp2
```

```
        ON w2.code = wp2.code
        WHERE w2.power = w.power
            AND wp2.age = wp.age
            AND wp2.is_evil = 0
    )
ORDER BY w.power DESC, wp.age DESC;
```

QUESTION 51. - Challenges

Julia asked her students to create some coding challenges. Write a query to print the hacker_id, name, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by hacker_id. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format: The following tables contain challenge data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- Challenges: The challenge_id is the id of the challenge, and hacker_id is the id of the student who created the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer

Sample Input 0

Hackers Table:

hacker_id	name
5077	Rose
21283	Angela
62743	Frank
88255	Patrick
96196	Lisa

Challenges Table:

challenge_id	hacker_id
61654	5077
58302	21283
40587	88255
29477	5077
1220	21283
69514	21283
46561	62743
58077	62743
18483	88255
76766	21283
52382	5077
74467	21283
33625	96196
26053	88255
42665	62743
12859	62743
70094	21283
34599	88255
54680	88255
61881	5077

Sample Output 0

```
21283 Angela 6
88255 Patrick 5
96196 Lisa 1
```

Sample Input 1

Hackers Table:

hacker_id	name
12299	Rose
34856	Angela
79345	Frank
80491	Patrick
81041	Lisa

Challenges Table:

challenge_id	hacker_id
63963	81041
63117	79345
28225	34856
21989	12299
4653	12299
70070	79345
36905	34856
61136	80491
17234	12299
80308	79345
40510	34856
79820	80491
22720	12299
21394	12299
36261	34856
15334	12299
71435	79345
23157	34856
54102	34856
69065	80491

Sample Output 1

```
12299 Rose 6
34856 Angela 6
79345 Frank 4
80491 Patrick 3
81041 Lisa 1
```

Explanation

For Sample Case 0, we can get the following details:

hacker_id	name	challenges_created
21283	Angela	6
88255	Patrick	5
5077	Rose	4
62743	Frank	4
96196	Lisa	1

Students 5077 and 62743 both created 4 challenges, but the maximum number of challenges created is 6 so these students are excluded from the result.

For Sample Case 1, we can get the following details:

hacker_id	name	challenges_created
12299	Rose	6
34856	Angela	6
79345	Frank	4
80491	Patrick	3
81041	Lisa	1

Students 12299 and 34856 both created 6 challenges. Because 6 is the maximum number of challenges created, these students are included in the result.

ANSWER:

```
SELECT
    H.hacker_id,
    h.name,
    COUNT(c.challenge_id) AS total_challenges
FROM Hackers h
JOIN Challenges c
ON h.hacker_id = c.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_challenges = (
        SELECT MAX(cnt)
```

```
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges
            GROUP BY hacker_id
        ) AS t
    )
    OR total_challenges IN (
        SELECT cnt
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges
            GROUP BY hacker_id
        ) AS t
        GROUP BY cnt
        HAVING COUNT(*) = 1
    )
ORDER BY total_challenges DESC, h.hacker_id;
```

QUESTION 52. - Contest Leaderboard

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the hacker_id, name, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending hacker_id. Exclude all hackers with a total score of 0 from your result.

Input Format:

The following tables contain contest data:

- **Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.**

Column	Type
hacker_id	Integer
name	String

- **Submissions: The submission_id is the id of the submission, hacker_id is the id of the hacker who made the submission, challenge_id is the id of the challenge for which the submission belongs to, and score is the score of the submission.**

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Sample Input:

Hackers Table:

hacker_id	name
4071	Rose
4806	Angela
26071	Frank
49438	Patrick
74842	Lisa
80305	Kimberly
84072	Bonnie
87868	Michael
92118	Todd
95895	Joe

Submissions Table:

submission_id	hacker_id	challenge_id	score
67194	74842	63132	76
64479	74842	19797	98
40742	26071	49593	20
17513	4806	49593	32
69846	80305	19797	19
41002	26071	89343	36
52826	49438	49593	9
31093	26071	19797	2
81614	84072	49593	100
44829	26071	89343	17
75147	80305	49593	48
14115	4806	49593	76
6943	4071	19797	95
12855	4806	25917	13
73343	80305	49593	42
84264	84072	63132	0
9951	4071	49593	43
45104	49438	25917	34
53795	74842	19797	5
26363	26071	19797	29
10063	4071	49593	96

Sample Output

```
4071 Rose 191
4842 Lisa 174
84072 Bonnie 100
4806 Angela 89
26071 Frank 85
80305 Kimberly 67
49438 Patrick 43
```

Explanation

Hacker 4071 submitted solutions for challenges 19797 and 49593, so the total score = 95 + max(43, 96) = 191.

Hacker 74842 submitted solutions for challenges 19797 and 63132, so the total score = max(98, 5) + 76 = 174

Hacker 84072 submitted solutions for challenges 49593 and 63132, so the total score = 100 + 0 = 100.

The total scores for hackers 4806, 26071, 80305, and 49438 can be similarly calculated.

ANSWER:

```sql
SELECT h.hacker_id, h.name, SUM(t.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id, challenge_id, MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) t
ON h.hacker_id = t.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_score > 0
ORDER BY total_score DESC, h.hacker_id;
```

Step: 1 Understand the main rule

A hacker can submit many times for the same challenge.

But the problem says:

1. For each challenge, only the maximum score is counted.

Example idea:

- Hacker submits scores → 20, 50, 30 for the same challenge
- We only consider → 50

So first rule:

For every hacker and every challenge → take MAX(score).

Step: 2 Calculate score per challenge

Think like this:

For each combination of: hacker_id + challenge_id

Find → **MAX(score)**

This gives the best score the hacker achieved for that challenge.

Step: 3 Calculate total score of the hacker

After finding the best score for each challenge, we must add them.

Formula: total_score = SUM(max_score_of_each_challenge)

Meaning: challenge1 max score + challenge2 max score + challenge3 max score

This becomes the total score of the hacker.

Display required columns

The output must contain:

Hacker_id ,Name, Total_score

The name comes from the Hackers table, so we join that table.

Step 4 Remove unnecessary hackers

The question says:

Exclude hackers whose total_score = 0

So we filter them out.

Step 5: Sort the result

Sorting rule:

1. Higher total_score first

   ORDER BY total_score DESC

2. If two hackers have the same score

   ORDER BY hacker_id ASC

STEP 6: Final thinking flow (very important)

Always solve it in this order:

1. Find MAX(score) for each challenge of each hacker
2. Add those scores → SUM(max_score)
3. Join with Hackers table to get name
4. Remove total_score = 0
5. Sort by total_score DESC, hacker_id ASC

—----------

QUESTION 53. - SQL Project Planning

You are given a table, Projects, containing three columns: Task_ID, Start_Date and End_Date. It is guaranteed that the difference between the End_Date and the Start_Date is equal to 1 day for each row in the table.

Column	Type
Task_ID	Integer
Start_Date	Date
End_Date	Date

If the End_Date of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.
Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

Sample Output

```
2015-10-28 2015-10-29
2015-10-30 2015-10-31
2015-10-13 2015-10-15
2015-10-01 2015-10-04
```

Explanation

The example describes following four projects:

- Project 1: Tasks 1, 2 and 3 are completed on consecutive days, so these are part of the project. Thus start date of project is 2015-10-01 and end date is 2015-10-04, so it took 3 days to complete the project.
- Project 2: Tasks 4 and 5 are completed on consecutive days, so these are part of the project. Thus, the start date of project is 2015-10-13 and end date is 2015-10-15, so it took 2 days to complete the project.
- Project 3: Only task 6 is part of the project. Thus, the start date of project is 2015-10-28 and end date is 2015-10-29, so it took 1 day to complete the project.
- Project 4: Only task 7 is part of the project. Thus, the start date of project is 2015-10-30 and end date is 2015-10-31, so it took 1 day to complete the project.

ANSWER:

```sql
SELECT start.Start_Date, MIN(end.End_Date)
FROM
    (
        SELECT Start_Date
        FROM Projects
        WHERE Start_Date NOT IN(SELECT End_Date FROM Projects)
    ) AS start
JOIN
    (
        SELECT End_Date
        FROM Projects
        WHERE End_Date NOT IN(SELECT Start_Date FROM Projects)
    ) AS end
ON end.End_Date > start.Start_Date
GROUP BY
    start.Start_Date
ORDER BY
    DATEDIFF(MIN(end.End_Date), start.Start_Date),
    start.Start_Date;
```

Step-by-Step Explanation

Step 1 – Find Project Start Dates

SELECT Start_Date FROM Projects

WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)

Meaning: Find Start_Date values that never appear as End_Date

Why? Because if a start date is not an end date, it means no task ended before it → so it must be a new project start.

Step 2 – Find Project End Dates

SELECT End_Date FROM Projects

WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)

Meaning: Find End_Date values that never appear as Start_Date.

Why? Because if no task starts after it, then the project ends there.

Step 3 – Join Start and End

ON end.End_Date > start.Start_Date

Meaning: Match every project start with end dates that occur after it.

Step 4 – Take Nearest End

MIN(end.End_Date)

Step 5 – Group By

GROUP BY start.Start_Date

We group results for each project start.

Step 6 – Order Results

ORDER BY DATEDIFF(MIN(end.End_Date), start.Start_Date), start.Start_Date

First sort by project duration *DATEDIFF(end , start)*

If days are same → sort by start date.

One Easy Memory Trick

To solve this problem remember only:

Project Start = Start_Date NOT IN End_Date

Project End = End_Date NOT IN Start_Date

Then

Join → take MIN(end)

QUESTION 54. - Placements

You are given three tables: Students, Friends and Packages. Students contains two columns: ID and Name. Friends contains two columns: ID and Friend_ID (ID of the ONLY best friend). Packages contains two columns: ID and Salary (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

Sample Input

ID	Friend_ID
1	2
2	3
3	4
4	1

Friends

ID	Name
1	Ashley
2	Samantha
3	Julia
4	Scarlet

Students

ID	Salary
1	15.20
2	10.06
3	11.55
4	12.12

Packages

Sample Output

```
Samantha
Julia
Scarlet
```

Explanation: See the following table:

ID	1	2	3	4
Name	Ashley	Samantha	Julia	Scarlet
Salary	15.20	10.06	11.55	12.12
Friend ID	2	3	4	1
Friend Salary	10.06	11.55	12.12	15.20

Now,

- **Samantha's best friend got offered a higher salary than her at 11.55**
- **Julia's best friend got offered a higher salary than her at 12.12**
- **Scarlet's best friend got offered a higher salary than her at 15.2**
- **Ashley's best friend did NOT get offered a higher salary than her**

The name output, when ordered by the salary offered to their friends, will be:

- **Samantha**
- **Julia**
- **Scarlet**

ANSWER:

```
SELECT s.Name
FROM Students AS s
    JOIN Friends AS f
ON s.ID = f.ID
    JOIN Packages AS p1
ON s.ID = p1.ID
    JOIN Packages AS p2
ON f.Friend_ID = p2.ID
    WHERE p2.Salary > p1.Salary
    ORDER BY p2.Salary;
```

QUESTION 55. - Symmetric Pairs

You are given a table, Functions, containing two columns: X and Y.

Column	Type
X	Integer
Y	Integer

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be symmetric pairs if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such symmetric pairs in ascending order by the value of X. List the rows such that $X_1 \leq Y_1$.

Sample Input

X	Y
20	20
20	20
20	21
23	22
22	23
21	20

Sample Output

```
20 20
20 21
22 23
```

ANSWER

```
-- X1 = Y2 and X2 = Y1
-- X1 <= Y1
-- List the rows such that X1 <= Y1
SELECT f1.X, f1.Y
FROM Functions AS f1
JOIN Functions AS f2
ON f1.X = f2.Y AND f2.X = f1.Y
WHERE f1.X <= f1.Y
GROUP BY f1.X, f1.Y
HAVING COUNT(*) > 1 OR f1.X < f1.Y
ORDER BY f1.X;
```

QUESTION 56. - Print Prime Numbers

Write a query to print all prime numbers less than or equal to 1000. Print your result on a single line, and use the ampersand (&) character as your separator (instead of a space). For example, the output for all prime numbers <= 10 would be:

2&3&5&7

ANSWER:

```
WITH RECURSIVE numbers AS (
   SELECT 2 AS n
    UNION ALL
    SELECT n + 1 FROM numbers WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM numbers
WHERE NOT EXISTS (
    SELECT 1 FROM numbers d WHERE d.n < numbers.n
    AND d.n > 1
    AND numbers.n % d.n = 0
);
```

—----------

QUESTION 57. - Interviews

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the contest_id, hacker_id, name, and the sums of total_submissions, total_accepted_submissions, total_views, and total_unique_views for each contest sorted by contest_id. Exclude the contest from the result if all four sums are 0.

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds 1 screening contest.

Input Format: The following tables hold interview data:

- **Contests: The contest_id is the id of the contest, hacker_id is the id of the hacker who created the contest, and name is the name of the hacker.**

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- **Colleges: The college_id is the id of the college, and contest_id is the id of the contest that Samantha used to screen the candidates.**

Column	Type
college_id	Integer
contest_id	Integer

- **Challenges: The challenge_id is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and college_id is the id of the college where the challenge was given to candidates.**

Column	Type
challenge_id	Integer
college_id	Integer

- **View_Stats: The challenge_id is the id of the challenge, total_views is the number of times the challenge was viewed by candidates, and total_unique_views is the**

number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- **Submission_Stats: The challenge_id is the id of the challenge, total_submissions is the number of submissions for the challenge, and total_accepted_submission is the number of submissions that achieved full scores.**

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Sample Input

Contests Table:

contest_id	hacker_id	name
66406	17973	Rose
66556	79153	Angela
94828	80275	Frank

Colleges Table:

college_id	contest_id
11219	66406
32473	66556
56685	94828

Challenges Table:

challenge_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

View_Stats Table:

challenge_id	total_views	total_unique_views
47127	26	19
47127	15	14
18765	43	10
18765	72	13
75516	35	17
60292	11	10
72974	41	15
75516	75	11

Submission_Stats Table:

challenge_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

Sample Output

```
66406 17973 Rose 111 39 156 56
66556 79153 Angela 0 0 11 10
94828 80275 Frank 150 38 41 15
```

Explanation

The contest 66406 is used in the college 11219. In this college 11219, challenges 18765 and 47127 are asked, so from the view and submission stats:

- **Sum of total submissions = 27 + 56 + 28 = 111**
- **Sum of total accepted submissions = 10 + 18 + 11 = 39**
- **Sum of total views = 43 + 72 + 26 + 15 = 156**
- **Sum of total unique views = 10 + 13 + 19 + 14 = 56**

Similarly, we can find the sums for contests 66556 and 94828.

ANSWER:

```
SELECT
    C.contest_id,
    C.hacker_id,
    c.name,
    SUM(ss.total_submissions) AS total_submissions,
    SUM(ss.total_accepted_submissions) AS
Total_accepted_submissions,
    SUM(vs.total_views) AS total_views,
    SUM(vs.total_unique_views) AS total_unique_views
FROM Contests c
JOIN Colleges cl ON c.contest_id = cl.contest_id
JOIN Challenges ch ON cl.college_id = ch.college_id
LEFT JOIN (
    SELECT challenge_id,
           SUM(total_views) AS total_views,
           SUM(total_unique_views) AS total_unique_views
    FROM View_Stats
    GROUP BY challenge_id
) vs ON ch.challenge_id = vs.challenge_id
LEFT JOIN (
```

```
    SELECT challenge_id,
           SUM(total_submissions) AS total_submissions,
           SUM(total_accepted_submissions) AS
Total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) ss ON ch.challenge_id = ss.challenge_id
GROUP BY c.contest_id, c.hacker_id, c.name
HAVING
    SUM(ss.total_submissions) +
    SUM(ss.total_accepted_submissions) +
    SUM(vs.total_views) +
    SUM(vs.total_unique_views) > 0
ORDER BY c.contest_id;
```

QUESTION 58. - 15 Days of Learning SQL

Julia conducted a 15 days of learning SQL contest. The start date of the contest was March 01, 2016 and the end date was March 15, 2016.

Write a query to print total number of unique hackers who made at least 1 submission each day (starting on the first day of the contest), and find the hacker_id and name of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest hacker_id. The query should print this information for each day of the contest, sorted by the date.

Input Format: The following tables hold contest data:

- **Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.**

Column	Type
hacker_id	Integer
name	String

- **Submissions: The submission_date is the date of the submission, submission_id is the id of the submission, hacker_id is the id of the hacker who made the**

submission, and score is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was March 06, 2016.

Hackers Table:

hacker_id	name
15758	Rose
20703	Angela
36396	Frank
38289	Patrick
44065	Lisa
53473	Kimberly
62529	Bonnie
79722	Michael

Submissions Table:

submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

Sample Output

```
2016-03-01 4 20703 Angela
2016-03-02 2 79722 Michael
2016-03-03 2 20703 Angela
2016-03-04 2 20703 Angela
2016-03-05 1 36396 Frank
2016-03-06 1 20703 Angela
```

Explanation

On March 01, 2016 hackers 20703, 36396, 53473, and 79722 made submissions. There are 4 unique hackers who made at least one submission each day. As each hacker made one submission, 20703 is considered to be the hacker who made the maximum number of submissions on this day. The name of the hacker is Angela.

On March 02, 2016 hackers 15758, 20703, and 79722 made submissions. Now 20703 and 79722 were the only ones to submit every day, so there are 2 unique hackers who made at least one submission each day. 79722 made 2 submissions, and the name of the hacker is Michael.

On March 03, 2016 hackers 20703, 36396, and 79722 made submissions. Now 20703 and 79722 were the only ones, so there are 2 unique hackers who made at least one submission each day. As each hacker made one submission, 20703 is considered to be the hacker who made the maximum number of submissions on this day. The name of the hacker is Angela.

On March 04, 2016 hackers 20703, 44065, 53473, and 79722 made submissions. Now 20703 and 79722 only submitted each day, so there are 2 unique hackers who made at least one submission each day. As each hacker made one submission, 20703 is considered to be the hacker who made the maximum number of submissions on this day. The name of the hacker is Angela.

On March 05, 2016 hackers 20703, 36396, 38289, and 62529 made submissions. Now 20703 only submitted each day, so there is 1 unique hacker who made at least one submission each day. 36396 made 2 submissions and the name of the hacker is Frank.

On March 06, 2016 only 20703 made a submission, so there is 1 unique hacker who made at least one submission each day. 20703 made 1 submission and the name of the hacker is Angela.

ANSWER:

```sql
SELECT
    S.submission_date,
    (
        SELECT COUNT(DISTINCT hacker_id)
        FROM Submissions AS s2
        WHERE s2.submission_date = s.submission_date
        AND (
            SELECT COUNT(DISTINCT submission_date)
            FROM Submissions AS s3
            WHERE s3.hacker_id = s2.hacker_id
            AND s3.submission_date <= s.submission_date
        ) = DATEDIFF(s.submission_date,'2016-03-01') + 1
    ) AS unique_hackers,
    (
        SELECT hacker_id
        FROM Submissions AS s4
        WHERE s4.submission_date = s.submission_date
        GROUP BY hacker_id
        ORDER BY COUNT(*) DESC, hacker_id
        LIMIT 1
    ) AS hacker_id,
    (
        SELECT name
        FROM Hackers
        WHERE hacker_id =
        (
            SELECT hacker_id
            FROM Submissions AS s5
            WHERE s5.submission_date = s.submission_date
            GROUP BY hacker_id
            ORDER BY COUNT(*) DESC, hacker_id
```

```
            LIMIT 1
        )
    ) AS name
FROM
    (SELECT DISTINCT submission_date FROM Submissions) AS s
ORDER BY s.submission_date;
```

SQL INTERVIEW QUESTIONS AND ANSWERS (IMPORTANT TOPICS)

==========

1 SQL Basics - Core commands every interview expects.

DDL (Data Definition Language)

- CREATE, ALTER, DROP, TRUNCATE

DML (Data Manipulation Language)

- INSERT, UPDATE. DELETE

DCL (Data Control Language)

- GRANT, REVOKE

TCL (Transaction Control Language)

- COMMIT, ROLLBACK, SAVEPOINT

Basic Query Clauses

- SELECT, WHERE, DISTINCT, ORDER BY

---

2 Joins - Very common interview topic.

Types:

- INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, SELF JOIN, CROSS JOIN

---

3 GROUP BY & Aggregations - Used for reporting and analytics.

Functions:

- COUNT(), SUM(), AVG(), MIN(), MAX()

---

4 HAVING

Used to filter grouped data.

Example

```
SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

---

5 Keys & Constraints - Database structure concepts.

Types:

- PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT

Important concepts:

- Difference between PRIMARY KEY and UNIQUE
- Multiple foreign keys in a table

---

6 SQL Functions - Aggregation Functions

- COUNT, SUM, AVG, MIN, MAX

String Functions

- UPPER(), LOWER(), LENGTH(), SUBSTRING(), TRIM()

Date Functions

- CURRENT_DATE / NOW(), EXTRACT(), DATEDIFF(), DATE_ADD()

Example

```
SELECT *
FROM employees
WHERE join_date > CURRENT_DATE - INTERVAL 30 DAY;
```

---

7 Subqueries - Query inside another query.

Types:

- Subquery in WHERE, Subquery in FROM, Subquery in SELECT, Correlated Subqueries

Example

```
SELECT name
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

---

8 Window Functions - Very important for HackerRank and advanced SQL interviews.

Functions:

- ROW_NUMBER(), RANK(), DENSE_RANK(), NTILE(), LAG(), LEAD()

Concepts:

- OVER(), PARTITION BY

Example

```
SELECT
    name, salary,
    RANK() OVER (ORDER BY salary DESC) AS rank_salary
FROM employees;
```

9️⃣ CASE Statement - Used for conditional logic.

Example

```
SELECT name,
        CASE
                WHEN salary > 100000
                        THEN 'High'
                WHEN salary > 50000
                        THEN 'Medium'
                ELSE 'Low'
        END AS salary_level
FROM employees;
```

🔟 Common Table Expressions (CTE) - Used to simplify complex queries.

Example

```
WITH salary_avg AS (
        SELECT AVG(salary) AS avg_salary
        FROM employees
)
SELECT * FROM employees
WHERE salary > (SELECT avg_salary FROM salary_avg);
```

1️⃣1️⃣ Views, Indexes & Stored Procedures

Views -> Virtual table created from a query.

Indexes-> Improve query performance.

Types: Clustered, Non-clustered

Stored Procedures vs Functions -> Used for reusable database logic.

1️⃣2️⃣ Transactions & ACID Properties

Transaction commands:

- **COMMIT, ROLLBACK, SAVEPOINT**

ACID properties:

- **Atomicity, Consistency, Isolation, Durability**

1️⃣3️⃣ Normalization

Database design principles.

Forms:

- **1NF, 2NF, 3NF, BCNF**

Also understand:

- **Denormalization**

1️⃣4️⃣ Common SQL Problem Patterns

Frequently asked practical problems.

- **Second highest salary**
- **Duplicate records**
- **Top N records**
- **Consecutive dates**
- **Running totals**
- **Gap and island problems**
- **Employees without department**
- **Department-wise max salary**
- **Delete duplicate rows**

🔁 Most Repeated Interview Questions

Basic

- Difference between DELETE, TRUNCATE, DROP
- Difference between WHERE and HAVING
- What is NULL

Joins

- Difference between INNER JOIN and OUTER JOIN
- Employees with no department

Keys

- Difference between PRIMARY KEY and UNIQU
- Can a table have multiple foreign keys?

Query Problems

- Second highest salary
- Find duplicate records
- Department-wise max salary
- Delete duplicates

Advanced

- Correlated subquery
- Difference between RANK() and DENSE_RANK()
- What is indexing?
- What is a VIEW?
- Explain ACID properties